

University of Edinburgh United Kingdom | Sep 2025 – Sep 2026 Expected

MSc High Performance Computing with Data Science

University of Liverpool United Kingdom | Sep 2023 – Jun 2025

BSc Computer Science, Xi'an Jiaotong-Liverpool University 2+2 Programme, First Class Honours

Technical Skills

Programming: C/C++, Python, SQL; basic JavaScript, HTML, CSS.

HPC: MPI, OpenMP, PETSc, Slurm, ARCHER2, hybrid MPI+OpenMP, strong / weak scaling, speedup, parallel efficiency, Linaro MAP.

Performance & Systems: process / thread models, synchronization overhead, communication overhead, cache locality, NUMA, memory bandwidth, file I/O, Linux.

Data & Visualization: Pandas, NumPy, Matplotlib, log parsing, CSV processing, scalability plots, performance reports.

AI / Agent / Backend: PyTorch, RAG, embeddings, FAISS / Chroma, LangChain / LlamaIndex, FastAPI, REST API, SQLite / MySQL, Redis, Docker, Git, Bash.

Projects

HPC Benchmark and Performance Analysis System for PETSc Parallel Solvers

GitHub: <https://github.com/lionleepower/petsc-benchmark-suite>

Project Description: Built a reproducible HPC benchmark workflow for evaluating the parallel performance of PETSc linear solvers. The project compares MPI-only and hybrid MPI+OpenMP execution modes on ARCHER2 and analyzes how problem size, core count, MPI ranks, and OpenMP threads affect solver performance. The workflow covers Slurm job scheduling, PETSc parameter configuration, MPI / OpenMP experiments, log parsing, metric calculation, performance visualization, and Linaro MAP profiling.

Tech Stack: C/C++, Python, PETSc, MPI, OpenMP, Slurm, Bash, Pandas, NumPy, Matplotlib, Linux, ARCHER2, Linaro MAP

Responsibilities:

- Built an automated PETSc benchmark workflow covering program compilation, parameter configuration, Slurm job submission, log collection, result parsing, metric calculation, and performance visualization to improve reproducibility and maintainability of large-scale experiments.
 - Designed and implemented Slurm batch scripts supporting MPI-only and hybrid MPI+OpenMP experiments across different nodes, MPI ranks, and OpenMP thread configurations.
 - Conducted PETSc linear solver performance tests on ARCHER2, covering 2D / 3D stencil benchmark problems with up to 40,960,000 unknowns and 128 cores.
 - Developed Python log parsing and data cleaning scripts to extract key fields from raw outputs, including problem size, nodes, ranks, threads, runtime, solver status, KSP iteration information, and error logs.
 - Used Pandas and Matplotlib to process experiment results, calculate runtime, speedup, and parallel efficiency, and generate scalability comparison plots across different problem sizes and core counts.
 - Compared MPI-only and hybrid MPI+OpenMP execution modes, analyzing performance differences from the perspectives of MPI communication overhead, OpenMP thread scheduling overhead, synchronization overhead, load balancing, and memory locality.
 - Used Linaro MAP to profile runtime bottlenecks, including computation time, MPI communication time, OpenMP region percentage, thread utilization, and potential load imbalance.
 - Applied strong scaling analysis, Amdahl's Law, process / thread models, cache locality, NUMA effects, and memory bandwidth considerations to explain parallel efficiency degradation.
 - Documented experiment configuration, execution workflow, and performance analysis results, making the benchmark process reproducible and extensible to additional PETSc examples, matrix formats, problem sizes, and parallel configurations.
-

AI Agent System for HPC Benchmark Performance Analysis

Project Description: Developed an AI Agent system for high-performance computing benchmark analysis based on PETSc

benchmark logs, performance metrics, and experiment documentation. The system supports Slurm / PETSc log ingestion, structured performance data storage, historical experiment querying, RAG-based HPC knowledge retrieval, Agent tool calling, Redis caching, and Markdown performance report generation. The goal is to combine HPC benchmark data, experiment documentation, and intelligent analysis workflows into an extensible AI Infrastructure tool for understanding MPI-only and hybrid MPI+OpenMP performance differences.

Tech Stack: Python, FastAPI, SQLite / MySQL, Redis, Pandas, Matplotlib, FAISS / Chroma, LangChain / LlamaIndex, Markdown, REST API, Linux, Git, Docker

Responsibilities:

- Designed the overall AI Agent performance analysis architecture, dividing the system into API layer, experiment data layer, log parsing layer, RAG retrieval layer, Agent tool calling layer, cache layer, report generation layer, and observability module.
- Built FastAPI backend interfaces for experiment log ingestion, natural language queries, historical result retrieval, performance metric calculation, Markdown report generation, and health checks, with unified request validation, exception handling, and response formatting.
- Implemented Slurm / PETSc log parsing modules to extract problem size, nodes, MPI ranks, OpenMP threads, total cores, runtime, execution mode, solver status, and error messages from raw outputs and convert them into structured experiment records.
- Designed database schemas to store benchmark configurations, execution status, performance metrics, error information, and generated reports, supporting queries by problem size, execution mode, total cores, and creation time.
- Designed indexing strategies for common query patterns, such as finding the best configuration by problem size and core count, comparing MPI-only and hybrid MPI+OpenMP results, and tracking experiment history by time range.
- Implemented performance metric calculation modules with Pandas, supporting automatic calculation of runtime, speedup, and parallel efficiency, and grouping results by problem size, nodes, ranks, threads, and execution mode.
- Built a RAG retrieval module by indexing PETSc, MPI, OpenMP, Slurm, Linaro MAP, and personal experiment notes, including Markdown document chunking, embedding generation, vector index construction, and top-k semantic retrieval.
- Designed an Agent tool calling workflow that allows the system to automatically select database query, log parsing, metric calculation, knowledge retrieval, chart generation, and Markdown report generation tools according to the user's question.
- Introduced Redis caching for hot experiment queries, frequently used statistics, short-term session state, and query-hash-based analysis results, using TTL to control cache lifecycle and reduce repeated database queries, vector retrieval, and model calls.
- Generated a TraceID for each analysis request and recorded user questions, tool invocation chains, cache hits, database query latency, RAG retrieval results, and report generation latency to debug retrieval failures, tool errors, and performance bottlenecks.
- Addressed common Agent application issues, including long context windows, inaccurate retrieval results, repeated generation, stale cache data, tool invocation failures, and insufficiently explainable performance analysis conclusions.
- Generated Obsidian / GitHub-compatible Markdown performance reports, including experiment summaries, configuration comparison tables, scalability analysis, bottleneck explanations, and follow-up optimization suggestions.

Publication

Li, Leyan. *Convolutional Neural Networks Based Medical Image Analysis*. International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.